PYTHON PROJECT BRIEF

Expense Tracker Pro

A one-day, standalone Python project to practise practical programming skills

Project type	Command-line application (CLI)
Suggested level	Intermediate
Target build time	One-day MVP (approximately 8 focused hours)
Core technologies	Python 3, SQLite, JSON, CSV, standard library
Primary goal	Strengthen implementation skills through a realistic application

Project principle: Build the application yourself. Use documentation and targeted help when needed, but avoid copying a complete solution. The purpose is to discover which Python concepts you can implement independently and which need more practice.

Prepared as a practical learning brief

1. Project overview

Build a terminal-based personal finance tracker that lets a user record income and expenses, organize transactions by category, set monthly budgets, search records, view summaries, and export reports. The program should store its data persistently so records remain available after the application closes.

The application is intentionally a CLI rather than a Django or GUI application. This keeps the focus on Python itself: data structures, functions, classes, files, databases, modules, exceptions, testing, and code organization.

Main menu – example

```
==================================
PERSONAL EXPENSE TRACKER
==================================
1. Add Income
2. Add Expense
3. View Transactions
4. Search Transactions
5. Set Monthly Budget
6. View Financial Summary
7. Export Report
8. Exit

Enter your choice:
```

2. Functional requirements

Feature	What it should do
Add income	Record an income amount, date, category/source, and description.
Add expense	Record an expense amount, date, category, and description.
View transactions	Display saved transactions in a readable, consistent format.
Update/delete	Allow a user to edit or remove a transaction by its unique ID.
Search/filter	Filter by category, transaction type, date range, or keyword.
Monthly budget	Set a budget, calculate spending and remaining amount, and flag overspending.
Financial summary	Calculate total income, total expenses, balance, and category-wise spending.
Export report	Export selected transaction/report data to CSV.
Persistence	Store transactions and budgets in SQLite; keep optional app settings in JSON.

Expected behaviour and validation

- Reject non-numeric amounts, zero or negative transaction amounts, and invalid menu choices.
- Validate dates and required text fields; show a useful message instead of crashing.
- Use a unique transaction ID so records can be updated or deleted reliably.
- Treat income and expenses consistently when calculating the balance: income minus expenses.
- A budget warning should be shown when spending in a category exceeds its configured monthly limit.
- Handle an empty database gracefully: show a helpful message and zero-valued summaries.

3. Suggested project structure

Split responsibilities into modules rather than placing all code in one file. This is a suggested structure; keep it lean if the one-day deadline becomes tight.

```
ExpenseTracker/
■■■ main.py
■■■ models/
■ ■■■ __init__.py
■ ■■■ transaction.py
■ ■■■ budget.py
■■■ services/
■ ■■■ __init__.py
■ ■■■ expense_service.py
■ ■■■ report_service.py
■■■ database/
■ ■■■ __init__.py
■ ■■■ db.py
■■■ utils/
■ ■■■ __init__.py
■ ■■■ validators.py
■ ■■■ helpers.py
■■■ exports/
■■■ tests/
■■■ __init__.py
■■■ test_expenses.py
```

4. Suggested classes and responsibilities

Transaction

- Represents one income or expense record.
- Suggested fields: id, amount, category, transaction_type, date, description.
- Keep validation and display behaviour close to the data only where it makes the design clearer.

Budget

- Represents a spending limit for a category and month.
- Supports setting a limit, calculating remaining budget, and checking whether the limit has been exceeded.

ExpenseService

- Coordinates transaction operations: add, list, update, delete, search, and filter.
- Calculates balances and totals using data returned by the database layer.

ReportService

- Creates monthly summaries and category-wise spending reports.
- Exports report data to CSV.

Database module

- Creates and manages the SQLite connection and tables.
- Provides focused database operations; use parameterized SQL queries.
- Handles transaction commit and rollback appropriately.

Example starting point – Transaction class

```
class Transaction:
    def __init__(self, amount, category, transaction_type, date, description):
        self.amount = amount
        self.category = category
        self.transaction_type = transaction_type
        self.date = date
        self.description = description
```

This is only a starting example. Add an ID, sensible validation, and any methods you find useful. Do not force every operation into a class: use ordinary functions where they are simpler.

5. Python concepts practised

Python topic	Where it appears in the project
Lists and dictionaries	Hold and process transaction data; build summaries.
Sets and tuples	Unique categories, fixed values, and unpacking.
Comprehensions and built-ins	Filter, transform, count, sort, and aggregate records.
Functions and arguments	Separate input, validation, business logic, and output.
OOP	Model transactions/budgets and separate service responsibilities.
Exceptions	Handle invalid input, missing records, file errors, and database errors.
Files / pathlib	Manage export paths and application files.

Python topic	Where it appears in the project
JSON and CSV	Store settings and export reports.
Modules and packages	Organize code by responsibility and import reusable functions.
Standard library	Use datetime, pathlib, csv, json, collections, logging, uuid, and more.
SQLite / SQL	CRUD, parameterized queries, keys, transactions, commit and rollback.
Testing	Test calculations, validation, CRUD behaviour, and error cases.
Professional practices	Type hints, docstrings, PEP 8, logging, clean structure, and Git.

6. One-day development plan

Aim for a working minimum viable product by the end of the day. If time is limited, prioritize a complete, reliable core over advanced features.

Time	Work	Deliverable
Hour 1	Create project structure, define the transaction/budget model, and design the SQLite tables.	Runnable project skeleton and schema.
Hour 2	Implement core data structures and model/service logic.	Transaction and budget operations.
Hour 3	Implement create, read, update, and delete operations.	Working transaction CRUD.
Hour 4	Add validation, exception handling, and useful error messages.	Invalid inputs handled safely.
Hour 5	Connect SQLite and make records persist between runs.	Persistent data storage.
Hour 6	Add date/category summaries and CSV export.	Useful reports and export.
Hour 7	Write tests and test normal, boundary, and failure cases.	Verified core behaviour.
Hour 8	Refactor, add logging, documentation, and polish the CLI.	A clean, usable MVP.

7. Completion checklist

■ Create the project structure	■ Implement Transaction class
■ Implement Budget class	■ Add income
■ Add expenses	■ View all transactions
■ Update and delete transactions	■ Search and filter transactions
■ Implement category-wise summaries	■ Implement monthly budgets
■ Handle invalid inputs and exceptions	■ Save records using SQLite
■ Export CSV reports	■ Use JSON for configuration
■ Implement logging	■ Write unit tests
■ Document the project	

8. Optional advanced extensions

These are deliberately optional. First finish the core application. Then implement one or more extensions to practise advanced Python features.

Concept	Optional use
Decorators	Create a decorator that measures the execution time of selected functions.
Generators	Yield transactions one at a time when producing a large report.
Custom iterators	Create an iterator that walks through transactions in date order.

Concept	Optional use
functools	Cache a repeatedly requested summary where caching is appropriate.
re	Validate or parse a simple transaction reference/description format.
asyncio	Practise concurrent external data fetching in a separate experiment.
threading	Run a background task such as a simulated report-generation job.
API integration	Optionally fetch currency conversion rates from a suitable public API.

Do not add concurrency or external API calls just to make the project look advanced. They introduce complexity and are not necessary for the core expense tracker.

9. Quality and acceptance criteria

- The app starts from main.py and presents a clear menu.
- A user can add, view, update, delete, search, and filter transactions.
- Data persists after the program is closed and restarted.
- Income, expenses, balance, and budget calculations are correct.
- Invalid values and missing records produce understandable messages rather than an unhandled crash.
- Database queries use placeholders/parameters rather than string concatenation.
- CSV export produces a valid file that can be opened in a spreadsheet application.
- Core business logic has automated tests, including at least a few invalid-input cases.
- The code is separated into sensible modules and includes basic setup/run instructions.

10. Recommended working rules

1. Use standalone Python 3. Do not use Django or a GUI framework for this exercise.
2. Write the code yourself; do not ask AI to generate the entire application.
3. Implement one feature at a time and run it before moving forward.
4. When stuck, isolate the specific problem and consult documentation or ask for a focused explanation.
5. Use a virtual environment and record dependencies if you install any external package. The core project can be built with the Python standard library.
6. Keep the one-day scope realistic. A finished MVP is more valuable than many half-finished advanced features.

11. Suggested learning references

Python documentation: https://docs.python.org/3/
SQLite documentation: sqlite3 module
CSV documentation: csv module
Unit testing: unittest module

Final note

This project is a practical diagnostic exercise, not a promise that every Python topic can be mastered in one day. Completing the MVP should expose exactly which concepts you can implement independently and which need more deliberate practice. Submit individual modules or features for code review as you build.